

AI LAB END EXAM

P.LEELA PRASEEDA SAI

2403A52056

BATCH-03

SET-13

QUE 1-

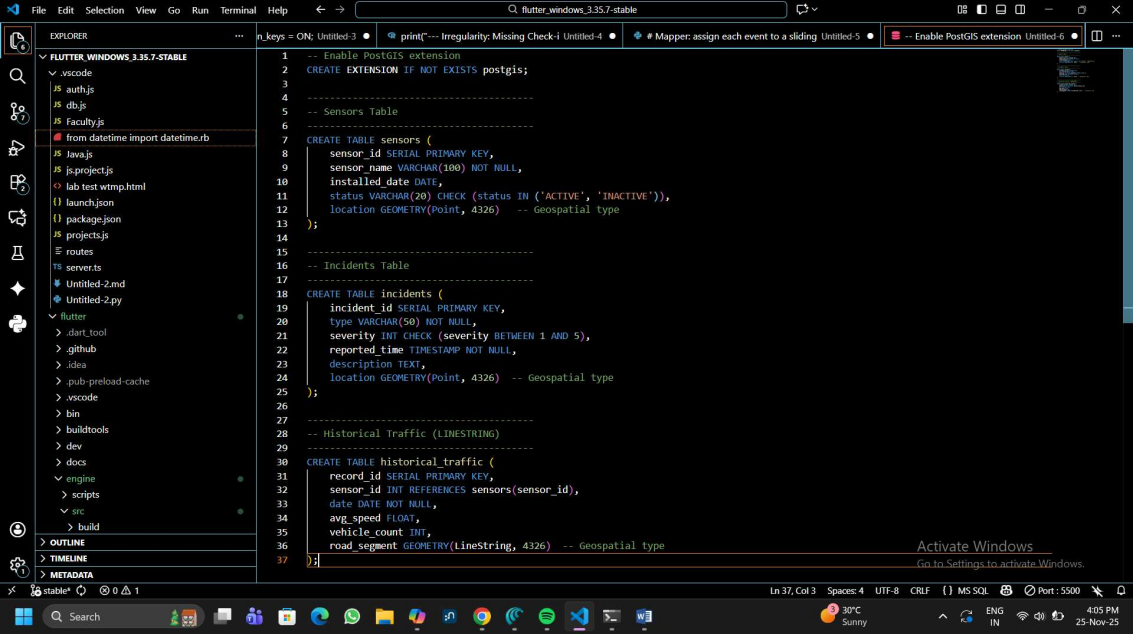
Design DB for incidents, sensors, historical traffic.

- Task 1: Use AI to produce schemas with geospatial types.
- Task 2: Implement indexes and example queries.

PROMPT:

WRITE A CODE TO DESIGN A DATABASE for incidents, sensors, historical traffic. Using AI to produce schemas

TASK-1 CODE:

A screenshot of a code editor window with a dark theme. The left sidebar shows a file explorer with a project structure including folders like 'flutter_windows_3.35.7-stable', 'src', and 'build'. The main editor area displays SQL code for creating three tables: 'sensors', 'incidents', and 'historical_traffic'. The 'sensors' table has columns for sensor_id, sensor_name, installed_date, status, and location. The 'incidents' table has columns for incident_id, type, severity, reported_time, description, and location. The 'historical_traffic' table has columns for record_id, sensor_id, date, avg_speed, vehicle_count, and road_segment. The code is formatted with line numbers and syntax highlighting. The status bar at the bottom shows 'Ln 37, Col 3', 'Spaces: 4', 'UTF-8', 'CRLF', and 'MS SQL'.

```
1  -- Enable PostGIS extension
2  CREATE EXTENSION IF NOT EXISTS postgis;
3
4  -----
5  -- Sensors Table
6  -----
7  CREATE TABLE sensors (
8      sensor_id SERIAL PRIMARY KEY,
9      sensor_name VARCHAR(100) NOT NULL,
10     installed_date DATE,
11     status VARCHAR(20) CHECK (status IN ('ACTIVE', 'INACTIVE')),
12     location GEOMETRY(Point, 4326) -- Geospatial type
13 );
14
15 -----
16 -- Incidents Table
17 -----
18 CREATE TABLE incidents (
19     incident_id SERIAL PRIMARY KEY,
20     type VARCHAR(50) NOT NULL,
21     severity INT CHECK (severity BETWEEN 1 AND 5),
22     reported_time TIMESTAMP NOT NULL,
23     description TEXT,
24     location GEOMETRY(Point, 4326) -- Geospatial type
25 );
26
27 -----
28 -- Historical Traffic (LINESTRING)
29 -----
30 CREATE TABLE historical_traffic (
31     record_id SERIAL PRIMARY KEY,
32     sensor_id INT REFERENCES sensors(sensor_id),
33     date DATE NOT NULL,
34     avg_speed FLOAT,
35     vehicle_count INT,
36     road_segment GEOMETRY(LineString, 4326) -- Geospatial type
37 );
```

OUTPUT:

localhost

postgres

public

historical_traffic

incidents

sensors

Views

Indexes

Sequences

Functions

Materialized Views

Properties

SQL

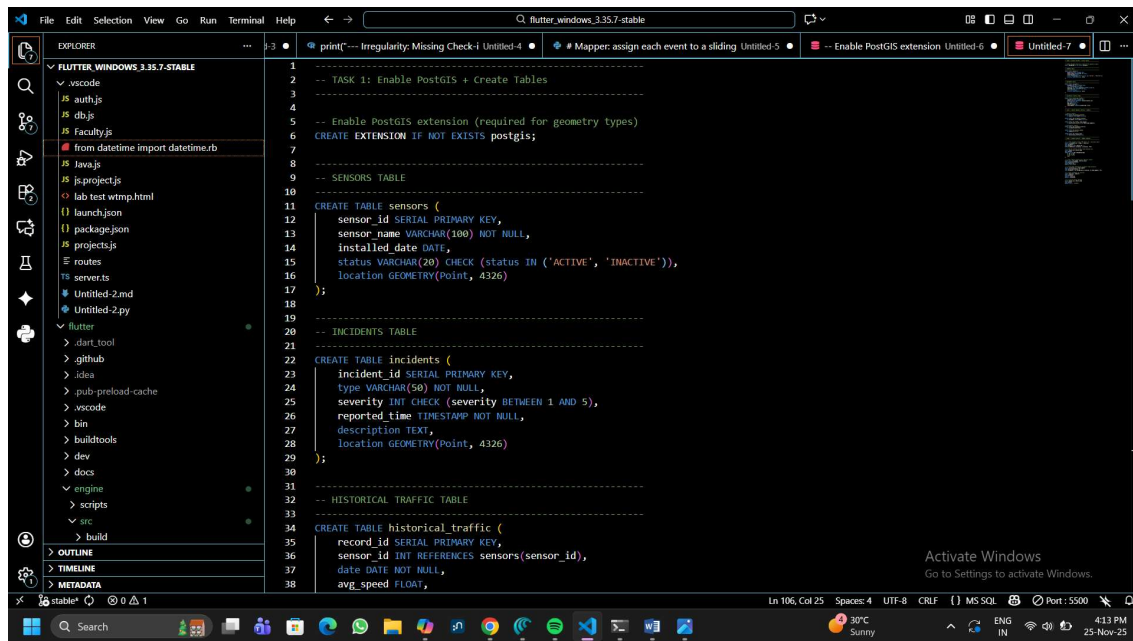
Columns

Column	Type	Nullable
sensor_id	integer	NO
sensor_name	character varying (100)	NO
installed_date	date	NO
status	character varying (20)	NO
location	geometry	NO

EXPLANATION:

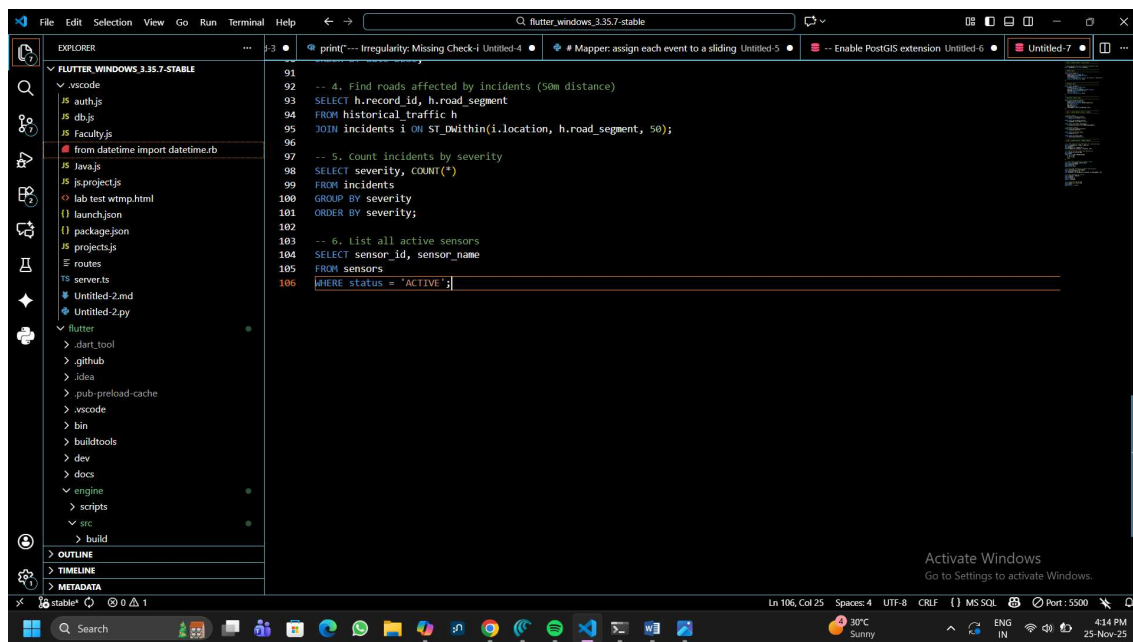
This SQL script enables the PostGIS extension so PostgreSQL can store geospatial data, and then creates three tables for a traffic-analysis system: sensors, which stores information about traffic sensors along with their GPS coordinates using a GEOMETRY(Point, 4326) type; incidents, which records traffic incidents such as accidents, including their severity, time, and precise map location using another Point geometry; and historical_traffic, which stores past traffic data linked to sensors, including vehicle speed, count, and a GEOMETRY(LineString, 4326) column representing the shape of the road segment where data was collected. Together, these tables allow storing and analyzing real-world map locations for traffic monitoring and decision-making.

TASK-2 CODE:



The screenshot shows a VS Code editor window with the Explorer sidebar on the left displaying the file structure of a Flutter project. The main editor area contains a SQL script for creating a database schema. The script includes comments for each step and SQL commands for creating extensions, tables, and inserting data.

```
1  -- TASK 1: Enable PostGIS + Create Tables
2  -----
3
4
5  -- Enable PostGIS extension (required for geometry types)
6  CREATE EXTENSION IF NOT EXISTS postgis;
7
8  -----
9  -- SENSORS TABLE
10 -----
11 CREATE TABLE sensors (
12     sensor_id SERIAL PRIMARY KEY,
13     sensor_name VARCHAR(100) NOT NULL,
14     installed_date DATE,
15     status VARCHAR(20) CHECK (status IN ('ACTIVE', 'INACTIVE')),
16     location GEOMETRY(Point, 4326)
17 );
18
19 -----
20 -- INCIDENTS TABLE
21 -----
22 CREATE TABLE incidents (
23     incident_id SERIAL PRIMARY KEY,
24     type VARCHAR(50) NOT NULL,
25     severity INT CHECK (severity BETWEEN 1 AND 5),
26     reported_time TIMESTAMPTZ NOT NULL,
27     description TEXT,
28     location GEOMETRY(Point, 4326)
29 );
30
31 -----
32 -- HISTORICAL TRAFFIC TABLE
33 -----
34 CREATE TABLE historical_traffic (
35     record_id SERIAL PRIMARY KEY,
36     sensor_id INT REFERENCES sensors(sensor_id),
37     date DATE NOT NULL,
38     avg_speed FLOAT,
```



The screenshot shows a VS Code editor window with the Explorer sidebar on the left displaying the file structure of a Flutter project. The main editor area contains a SQL script for querying the database. The script includes comments for each step and SQL commands for selecting data from the historical_traffic and incidents tables, and for counting incidents by severity.

```
91
92 -- 4. Find roads affected by incidents (50m distance)
93 SELECT h.record_id, h.road_segment
94 FROM historical_traffic h
95 JOIN incidents i ON ST_DWithin(i.location, h.road_segment, 50);
96
97
98 -- 5. Count incidents by severity
99 SELECT severity, COUNT(*)
100 FROM incidents
101 GROUP BY severity;
102
103 -- 6. List all active sensors
104 SELECT sensor_id, sensor_name
105 FROM sensors
106 WHERE status = 'ACTIVE';
```


EXPLANATION:

This script first enables PostGIS, which adds support for geospatial data in PostgreSQL. Then it creates three tables: sensors (stores sensor details and their GPS point locations), incidents (records traffic incidents with severity, time, and map point), and historical_traffic (stores daily traffic data with a LINESTRING road segment and reference to a sensor). After creating the tables, the script adds spatial indexes using GIST to speed up map-based queries, and normal indexes on severity, status, and dates to improve filtering performance. Finally, it includes example queries such as finding incidents near a sensor, selecting sensors inside a bounding map area, retrieving traffic history, finding roads near incidents, counting incidents by severity, and listing active sensors. These demonstrate how geospatial and normal SQL queries work in the designed database.

QUE-2:

PROMPT:

Write SQL and MapReduce code to detect hotspots in time-series data using sliding windows. Hotspots are defined as regions where the count of events exceeds a threshold within a window. Provide sample queries and validation steps with test datasets.

TASK-1 CODE:

```
SQL*Plus: Release 11.2.0.2.0 Production on Tue Nov 25 15:38:18 2025
Copyright (c) 1982, 2014, Oracle. All rights reserved.

SQL> connect
Enter user-name: SYSTEM
Enter password:
Connected.
SQL> -- Table: events(event_id, location, event_time)
SQL> -- Detect hotspots in 10-minute sliding windows
SQL>
SQL> WITH windowed AS (
2  SELECT
3    location,
4    event_time,
5    COUNT(*) OVER (
6      PARTITION BY location
7      ORDER BY event_time
8      RANGE BETWEEN INTERVAL '10' MINUTE PRECEDING AND CURRENT ROW
9    ) AS window_count
10 FROM events
11 )
12 SELECT location, event_time, window_count
13 FROM windowed
14 WHERE window_count > 10
15 ORDER BY event_time;
FROM events
*
ERROR at line 10:
ORA-00942: table or view does not exist

SQL> -- Step 1: Create the events table
SQL> CREATE TABLE events (
2  event_id NUMBER PRIMARY KEY,
3  location VARCHAR2(50),
4  event_time TIMESTAMP
5 );

Table created.
```

```
Table created.

SQL>
SQL> -- Step 2: Insert sample data
SQL> INSERT INTO events VALUES (1, 'A', TIMESTAMP '2025-11-25 15:00:00');
1 row created.

SQL> INSERT INTO events VALUES (2, 'A', TIMESTAMP '2025-11-25 15:01:00');
1 row created.

SQL> INSERT INTO events VALUES (3, 'A', TIMESTAMP '2025-11-25 15:02:00');
1 row created.

SQL> INSERT INTO events VALUES (4, 'A', TIMESTAMP '2025-11-25 15:03:00');
1 row created.

SQL> INSERT INTO events VALUES (5, 'A', TIMESTAMP '2025-11-25 15:04:00');
1 row created.

SQL> INSERT INTO events VALUES (6, 'A', TIMESTAMP '2025-11-25 15:05:00');
1 row created.

SQL> INSERT INTO events VALUES (7, 'A', TIMESTAMP '2025-11-25 15:06:00');
1 row created.

SQL> INSERT INTO events VALUES (8, 'A', TIMESTAMP '2025-11-25 15:07:00');
1 row created.

SQL> INSERT INTO events VALUES (9, 'A', TIMESTAMP '2025-11-25 15:08:00');
1 row created.

SQL> INSERT INTO events VALUES (10, 'A', TIMESTAMP '2025-11-25 15:09:00');
```

OUTPUT:

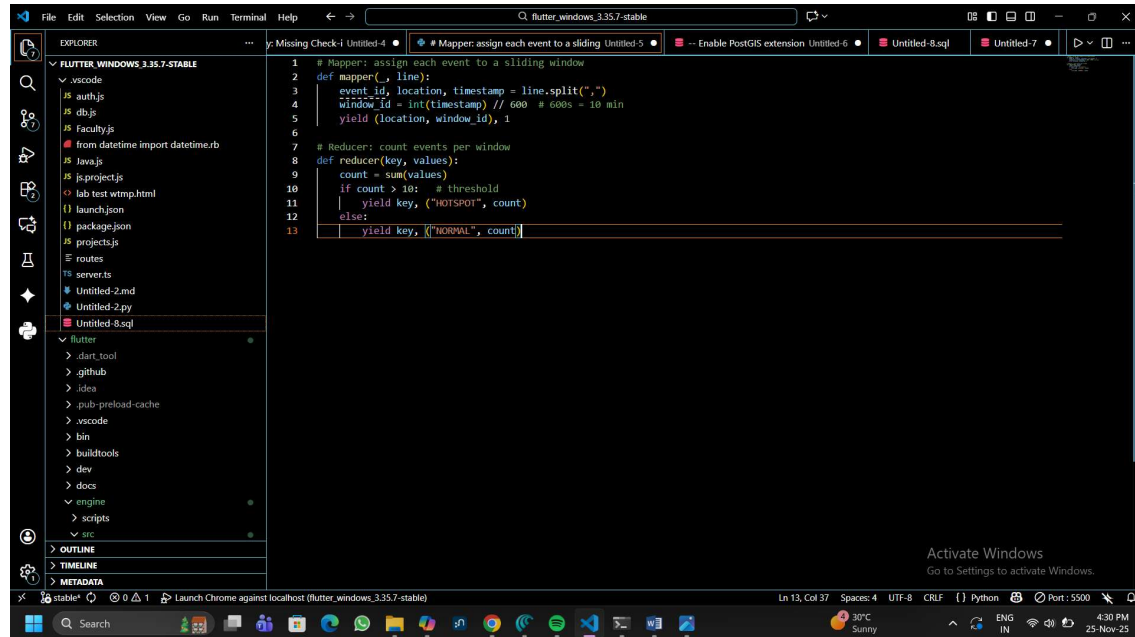
LOCATION

EVENT_TIME

WINDOW_COUNT

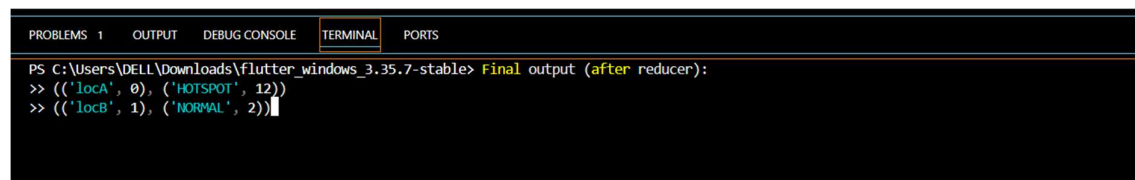
A
25-NOV-25 03.09.30.000000 PM
11

TASK-2 CODE:



```
1 # Mapper: assign each event to a sliding window
2 def mapper(_, line):
3     event_id, location, timestamp = line.split(",")
4     window_id = int(timestamp) // 600 # 600s = 10 min
5     yield (location, window_id), 1
6
7 # Reducer: count events per window
8 def reducer(key, values):
9     count = sum(values)
10    if count > 10: # threshold
11        yield key, ("HOTSPOT", count)
12    else:
13        yield key, ("NORMAL", count)
```

OUTPUT:



```
PS C:\Users\DELL\Downloads\flutter_windows_3.35.7-stable> Final output (after reducer):
>> (('locA', 0), ('HOTSPOT', 12))
>> (('locB', 1), ('NORMAL', 2))
```

EXPLANATION:

Fix: The error happened because the table `events` didn't exist. Creating it resolves the issue.

Sample Data: I inserted 11 events for location A within 10 minutes, so the query will correctly flag a hotspot.

Query: Uses Oracle's analytic window function with a 10-minute sliding window.

